SOFTWARE QUALITY ASSURANCE

Revision Questions — Answer Report (Simple English)

Based on the Case Study: Bank Management System (BMS)

Introduction

This report answers all 10 revision questions using easy, plain words. Each answer connects back to the Bank Management System (BMS) story: a bank (National Trust Bank) hired a company (ABC Software Solutions) to build a banking system, but the project had many problems — new features were added without proper approval, planning documents were incomplete, testing started late, the test environment did not match the real system, and many bugs were left unfixed. Because of this, the bank delayed the launch of the system.

Question 1: Software Testing Strategy

1. Four reasons every project needs a clear testing strategy

1. **It keeps testing consistent —** In BMS, different teams tested in different ways, so results could not be trusted. A clear plan means everyone tests the same way, every time.
2. **It focuses effort on what matters most —** A bank system must be very safe and correct. A strategy tells the team which parts (like money transfers and security) need the most testing.
3. **It helps with planning time and people —** BMS started testing too late. If the strategy is made early, the team can prepare the test environment, data, and staff on time.
4. **It gives proof that the system is good enough —** A strategy sets clear rules for what "done" and "good enough" look like, so managers can make a safe decision about releasing the system.

2. Three testing methods recommended for BMS, with reasons

5. **Risk-based testing —** Test the riskiest parts first and hardest — like money transfers, loans, and card payments — because mistakes there cause the most damage, as we saw with BMS's wrong balances and payment errors.
6. **Security testing —** BMS had a problem where people could reach banking functions they should not access. Testing security (logins, permissions, fraud checks) protects customers' money and data.
7. **Automated regression testing —** BMS had old bugs coming back after new fixes. Automated tests can quickly re-check the whole system every time something changes, so fixed bugs stay fixed.

Question 2: Project Documentation and Quality Assurance

1 & 2. Five important project documents, what they do, and why they matter

Document	What it is for (simple)	Why it helps
Software Requirements Specification (SRS)	Writes down exactly what the system must do (accounts, loans, transfers, security, rules).	Everyone — developers, testers, the bank — agrees on the same plan. This stops confusion and repeated work, which happened in BMS.
Software Development Plan (SDP)	Explains how the system will be built: steps, schedule, people, coding rules.	Keeps all teams building the same way. BMS teams used different coding styles, which made joining the parts together hard.
Software Quality Assurance Plan (SQAP)	Explains how quality will be checked: reviews, rules, and who is responsible.	Makes sure quality is checked all the way through the project, not just at the end, as happened in BMS.
Test Plan / Test Strategy Document	Explains what will be tested, how, when, and with what tools.	Stops testing from starting late or being disorganized, unlike in BMS.

Document	What it is for (simple)	Why it helps
Test Summary Report	Collects all test results and bug information at the end of testing.	Gives management clear facts to decide if the system is ready. BMS was missing a full report, which was one reason launch was delayed.

Together, these documents help people communicate clearly, keep track of what has been done, and give proof that the system is good enough to launch — things that were missing or incomplete in the BMS project.

Question 3: Contract Review

1. Four problems that Contract Review could have stopped

8. **Scope creep (too many extra features added) —** The bank kept adding new features (fingerprint login, fraud detection, mobile wallet, AI credit scoring) and the team just said yes without checking the effect. A Contract Review would check every new request first.
9. **Late delivery —** Adding features without re-checking the schedule made the 12-month deadline unrealistic. A Contract Review would re-plan the schedule whenever something new is added.
10. **Spending more money than planned —** New features cost extra money that was not in the budget. A Contract Review would check the cost first and get the bank to agree to pay for it.
11. **Disagreements between the bank and the developers —** Because changes were not written down and agreed properly, both sides had different ideas about what was included. A Contract Review keeps a clear written record everyone agrees to.

2. Three goals of Contract Review and how they help BMS succeed

12. **Make sure requirements are clear and complete —** This would catch all the bank's real needs early, so nothing important is missed or duplicated later.
13. **Check that the company can really deliver (time, money, skills) —** This would confirm the schedule and budget are realistic for everything the bank is asking for, before agreeing to it.
14. **Set up a proper way to handle changes —** This directly fixes BMS's biggest problem: changes were accepted without checking their effect on scope, cost, time, or risk. A change-control step keeps the project under control.

Question 4: Software Testing Implementation

As the Lead Software Test Engineer, here is a clear, step-by-step testing process for BMS, explained simply.

Test Planning

Decide what to test (accounts, loans, transfers, cards, online/mobile banking, compliance reports), what the biggest risks are, who will do the testing, and when.

Test Environment Setup

Build a test system that looks and works just like the real (production) system, including its connections to national payment networks. This fixes BMS's problem where the test system was different from the real one.

Test Data Preparation

Create realistic sample data — customer accounts, transactions, loans — including tricky cases like not enough money in an account or an expired card. This fixes BMS's problem of unrealistic test data.

Test Case Development

Write clear, detailed test steps for each feature, covering normal use, mistakes, and edge cases (e.g., two people trying to use the same account at once).

Test Execution

Run the tests — unit, integration, system, performance, security, and user acceptance testing — starting early instead of late, so problems are found while they are still easy and cheap to fix.

Defect Reporting and Tracking

Write down every bug clearly (what happened, how serious it is, which part of the system) and give it to a specific developer to fix. In BMS, some bugs were never given to anyone.

Retesting and Regression Testing

After a bug is fixed, test it again to make sure it's really fixed, and check that the fix didn't break anything else. This stops old bugs from coming back, which happened in BMS.

Test Reporting and Documentation

Keep clear, ongoing records of test results and bugs, and build a full Test Summary Report so management always knows the true quality status.

Test Closure

Confirm all planned tests are finished, check that there are no serious open bugs, and formally sign off that testing is complete before recommending the system for launch.

Question 5: Software Quality Metrics and Testing Evaluation

1. Five quality metrics/KPIs to check BMS testing

15. **Defect Density —** How many bugs are found per part of the system. A high number in money-transfer or security features is a warning sign.
16. **Defect Removal Efficiency —** What percentage of bugs are caught before launch, compared to after. A low number means testing is not catching enough problems early.
17. **Test Coverage —** How much of the requirements and code has actually been tested. BMS had incomplete test cases, so coverage would show the gaps.
18. **Number of Open Critical/High Bugs —** How many serious bugs are still unresolved. BMS was delayed because there were too many of these.
19. **Average Time to Find and Fix Bugs —** How long it takes to catch and repair a bug. Slow numbers here explain why some BMS bugs were still open at launch time.

2. How these numbers help managers decide on release

These numbers turn a guess ("it feels okay") into real proof. A manager can see if quality is getting better (fewer bugs, more tests passed, fewer serious bugs left) or getting worse. For BMS, seeing many serious bugs still open and low test coverage gives clear, honest proof that the system is not ready — which is exactly why the bank delayed the launch. The numbers also show the team where to focus their remaining time before trying again.

Question 6: Development and Quality Planning

1. Why an SDP and SQAP are important before development starts

BMS started without proper planning, and even the plans that existed (SDP and SQAP) were incomplete and not followed properly. This caused different teams to code differently, poor communication, and quality problems that were only found very late. A Software Development Plan (SDP) made early would give every team the same approach, schedule, and coding rules, making it much easier to join all the parts of BMS together. A Software Quality Assurance Plan (SQAP) made early would set out how quality will be checked all through the project — not

just at the end — so problems are caught sooner. Having both plans ready before work starts gives everyone the same clear picture, reducing confusion, wasted work, and the late discovery of serious bugs that delayed BMS's launch.

2. Six main differences between the SDP and the SQAP

Point	Software Development Plan (SDP)	Software Quality Assurance Plan (SQAP)
Main focus	How to build the software: steps, schedule, resources	How to check the software and process are good quality
Covers	The whole build and delivery process	Quality checks across the whole project (reviews, audits, rules)
Who owns it	Project Manager / Development Lead	Quality Assurance Manager / QA team
Main contents	Milestones, coding rules, people, tools, risks	Quality goals, standards, review schedule, bug-handling rules
What it's about	The product and the timeline (what and when)	The process (how quality is checked and proven)
How success is measured	Delivered on time and on budget	Proof that quality rules were followed and bugs controlled

Question 7: Test Environment and Test Data Management

1. Five features of a good test environment

20. **Matches the real (production) system —** Same hardware, software, and settings as the real system, so results can be trusted. BMS failed at this — its test system was very different.
21. **Properly connected to other systems —** For BMS, this means safely linking to national payment networks so full transactions can be tested from start to end.
22. **Stable and separate from other work —** The test system should only be used for testing, so results are not confused by unrelated changes.
23. **Proper security controls —** Since it's a bank, the test system should have realistic logins and permissions so security testing gives true results.
24. **Well managed and tracked —** Keep track of which version of the software and settings is being tested, so testing is always done against a known, clear setup.

2. Five good habits for preparing and managing test data

25. **Use data that looks like real banking data —** Realistic account balances, transaction history, and customer details, fixing BMS's problem of unrealistic test data.
26. **Hide or protect real customer information —** Mask sensitive details so testing does not break privacy rules or bank regulations.
27. **Include tricky and unusual cases, not just normal ones —** For example: not enough money, expired cards, two people using the same account at the same time.
28. **Keep test data updated and organized —** Update the data whenever requirements change, and keep track of different versions so tests can be repeated and compared fairly.
29. **Use tools to create and reset test data quickly —** Automatically generate large amounts of realistic data (for busy-time testing) and reset it easily between test runs.

Question 8: Defect Management and Risk Management

1. Big risks of launching BMS without proper quality checks

- **Money risk —** Wrong transactions and incorrect balances could cause real financial loss.

- **Security and fraud risk —** People could reach banking functions they shouldn't, opening the door to fraud or theft.
- **Legal and rule-breaking risk —** Missing proof that Central Bank rules were followed, and incomplete records, could lead to fines or losing the license to operate.
- **Day-to-day operation risk —** Slow performance during busy times could disrupt banking services for everyone.
- **Reputation risk —** Public mistakes, fraud, or system failures would badly damage the bank's reputation and customer trust.
- **Disaster-recovery risk —** Since backup procedures were never checked, the bank could lose data or suffer long outages if something goes wrong.
- **Legal liability risk —** Problems with data accuracy or privacy could lead to legal claims from customers or regulators.

2. How good bug management improves quality — stage by stage

30. **Defect Identification (finding bugs) —** Doing all the testing properly (including the security, performance, and user-acceptance testing that BMS skipped) actively finds problems before customers do.
31. **Defect Reporting (writing bugs down) —** Every bug is written clearly with steps to reproduce it, fixing BMS's problem of poorly documented bugs.
32. **Defect Tracking (following up on bugs) —** Every bug is given to a specific person and followed until it's fixed, fixing BMS's problem of bugs that were never assigned to anyone.
33. **Defect Resolution (fixing bugs) —** Developers fix the most serious and dangerous bugs first, like unauthorized access, before smaller ones.
34. **Defect Verification and Closure (checking the fix) —** Every fix is tested again, and related features are re-checked, so bugs don't come back — which is what happened repeatedly in BMS.

Question 9: Test Reporting and Software Release Decision

1. What a good Test Summary Report should include

- What was tested, and what was planned but not tested, with reasons why.
- A summary of results for each part of the system (accounts, loans, transfers, cards, security).
- How much of the system was actually tested (test coverage).
- Total number of bugs found, how serious they are, and their current status (open, fixed, checked, delayed).
- A list of any serious bugs still open, and how much damage they could cause.
- Results from special tests: performance under heavy load, security testing, and backup/disaster-recovery checks.
- Proof that legal and Central Bank rules have been followed.
- An overall opinion on quality and a clear recommendation on whether to launch or not.

2. What to think about before deciding to release the system

35. **How serious and how many bugs are still open —** Even one very serious bug (like unauthorized access) can be a reason to wait, as happened with BMS.
36. **How much money or business risk is involved —** Weighing the possible cost of known problems against the cost of delaying the launch.
37. **Whether legal and Central Bank rules are satisfied —** Checking there is written proof that all required rules have been followed.
38. **Whether the system is secure enough —** Confirming that security and fraud checks show customer money and data are properly protected.

39. **Whether the system works well in daily use —** Checking that performance is good at busy times and that backups/disaster recovery have been tested.
40. **Whether everyone agrees it's ready —** Getting formal agreement from business leaders, QA, and technical leaders, based on the Test Summary Report.

Question 10: Software Quality Improvement Plan

Here is a simple improvement plan to fix the weaknesses found in the BMS project.

Contract Review

Always formally check any new request or change before agreeing to it — how it affects time, cost, risk, and scope — and get both sides to sign off.

Software Development Planning

Create one clear, complete development plan for all teams, with the same coding rules and documentation style, so joining the parts together is easier.

Software Quality Assurance Planning

Finish and properly follow the quality plan, with regular quality checks at each stage — not just at the end.

Software Testing Strategy

Use a clear, risk-based testing plan that focuses on the most important parts (transactions, security, rules), agreed before development begins.

Software Testing Implementation

Start testing early, alongside development, using a realistic test environment and realistic data, and make sure all needed testing (including performance, security, and user-acceptance testing) is actually done.

Defect Management

Set up a clear process where every bug is written down, given to someone, fixed, and checked again before being closed.

Test Reporting and Documentation

Keep regular test status updates during the project, and produce one full, honest Test Summary Report before any launch decision.

Continuous Quality Improvement

Use the quality numbers from each stage (bug counts, test coverage, etc.) to keep improving the process, training, and standards for future work.

END OF REPORT